체크박스(☐)를 클릭하면 체크되고, Edge·크롬·Adobe Reader 에서 저장(Ctrl+S)하면 남습니다.

교과 7 산출물 정리와 PM 실행 계획

2026-10-06 · 작성 Sung Hyung Lee (PM) · 6인 팀 공용 · 기준 자료: 산출물 예시_교과7

한눈에 보기

교과 7 의 산출물은 11개이고, 대부분 Git 저장소 안의 정해진 위치에 파일로 제출합니다. 프로그램은 이미 완성되어 있으므로, 남은 일은 ① **900장 라벨링** ② **버그 수정과 부족한 부분 보완** ③ **문서 산출물 작성** 세 가지입니다. 이 중 1순위는 라벨링입니다.

항목	핵심
최종 결과	검수 완료 이미지 900장 + YOLO TXT 900개 (FINAL) → 교과 8 학습 데이터
제출 방식	Git 저장소 (코드·문서) + 캡처 증빙 + 발표 시연. 실제 JPG·TXT 900장은 Git 에 올리지 않음
최종 목표 숫자	Validation 오류 0 · FAIL 0 · 미처리 REVIEW 0 · Critical Error 0
팀 원칙	라벨링 1순위 · 코드는 담당자 2명만 수정 · 버그는 기록 → 수정 → 테스트 → 커밋 · 10/7 Freeze 이후 새 기능 금지

1. 산출물 예시 요약 (11개)

#	산출물	제출 형태	핵심 내용
1	라벨링 프로그램	프로그램 + 실행 캡처 2~3장 + 시연	열기 → 기존 BBox 확인 → 수정 → 저장 → 다음 → 돌아와 복원 확인 → Validation
2	소스코드	Git 저장소	기능별 코드, `requirements.txt`, `configs/classes.yaml`, 기능별 Commit 이력 (`feat:` `fix:` `docs:`)
3	FINAL YOLO 라벨	캡처 증빙	`data/final/images` 900장 + `labels` 900개, 폴더·파일 수 캡처, 대표 이미지 3~5장, 대표 TXT 2~3개
4	Project Baseline	`docs/project_baseline.md`	데이터 기준, 작업자·검수자, WORK/FINAL 위치, Git 규칙, 완료 기준
5	Class 기준서	`docs/class_guide.md`	Class 마다 대상 / 포함 / 제외 / 애매하면 REVIEW, Class 4 미사용
6	BBox 기준서	`docs/bbox_guide.md`	밀착, 경계에 걸림, 겹침, 작은 객체, 가림, 한 객체 = BBox 1개, REVIEW 사유
7	Dataset Manifest	`manifests/dataset_manifest.csv`	파일명, 출처, split, scene_type, 작업자, 상태, QA 상태, REVIEW 사유
8	QA Summary	`reports/qa_summary.md`	Validation 오류 종류·수 → Human QA → 최종 FAIL 0 / REVIEW 0

9	Test Report	`reports/test_report.md`	Golden → Pilot → Acceptance, 실패는 문제 → 원인 → 조치 → 재시험
10	README	`README.md`	설치, 실행, 사용 순서, 각 문서 위치 안내 (상세는 링크로)
11	교과 8 Handoff	`docs/subject08_handoff.md`	FINAL 수량, 형식, Class (4번 미사용), QA 결과, Manifest 위치

예시에서 특히 강조하는 것

- **QA Summary ≠ Test Report.** QA Summary 는 "라벨 데이터가 맞는가", Test Report 는 "프로그램이 제대로 동작하는가"입니다.
- **Test Report 에는 실패 기록이 있어야 합니다.** 처음부터 전부 PASS 일 필요가 없고, 오류를 찾아 고친 과정을 남기는 것이 중요합니다.
- **Git 이력은 기능 단위로.** `update`, `수정`, `최종` 같은 메시지는 쓰지 않습니다.
- **README 는 안내 페이지.** Class·BBox·QA 내용을 다시 쓰지 않고 해당 문서 위치로 연결합니다.
- **Empty TXT 는 무조건 오류가 아닙니다.** 정상 김치면 PASS, 대상이 있으면 누락 라벨로 수정합니다.

2. 우리 프로그램과 비교

이미 된 것

1 프로그램 기능 전부 · 2 기능별 코드와 `requirements.txt` · 3 FINAL 생성(`07_build_final.py`) · 7 Manifest · 8 QA Summary(`05`) · 9 Golden 테스트(`09`) · 10 README · 11 Handoff(`08`)

부족한 부분 (완료하면 체크)

부족한 것	지금 상태	해야 할 일
☐ `configs/classes.yaml` 없음	Class 표가 `config.py` 안에 코드로 있음	yaml 파일을 만들고 프로그램이 읽도록 수정 (**코드**)
☐ FINAL 폴더 구조가 다름	RAW 와 같은 구조 (`데이터 1/images/train/…`)	`final/images`, `final/labels` 평면 내보내기 추가, split 은 Manifest 에 보존 (**코드**)
☐ Manifest 형식·위치가 다름	`work/manifest.csv`, 열 이름 `assignee`, `issue` 등	`manifests/dataset_manifest.csv` 로 예시 열 형식에 맞춰 내보내기 (**코드**)
☐ `.gitignore` 가 `reports/` 전체 제외	보고서 md 가 Git 에 안 올라감	md 는 올리고 csv 만 제외하도록 수정
☐ 보고서 이름·위치가 다름	`QA_Summary.md`, `Handoff_교과8.md`	`reports/qa_summary.md`, `docs/subject08_handoff.md` 로 맞추기
☐ Git 저장소·커밋 이력 없음	zip 으로만 주고받음	오늘 저장소 생성, 지금부터 고치는 것마다 `fix:` / `feat:` 커밋
☐ 문서 4개 없음	Baseline, Class 기준서, BBox 기준서, Test Report	팀이 작성 (기준은 팀이 정해야 하는 내용)
☐ 캡처·시연 영상 없음	-	실제 데이터 화면 2~3장, 시연 2분 녹화

Git 이력은 **지금부터 실제로 고치는 내용을 그때그때 커밋**하면 자연스럽게 쌓입니다. 위 코드 보완과 버그 수정이 그대로 `feat:` / `fix:` 이력이 됩니다. 과거 이력을 꾸며 만들지 않습니다 (발표 질문에 설명할 수 없음).

3. 팀 운영 원칙 (오늘 시작 회의에서 공지)

1. **라벨링이 1순위.** 900장 품질이 최종 결과물입니다. 1인 150장은 역할과 관계없이 모두 합니다.
2. **코드는 담당자 2명만 수정** (GUI·저장 담당). 나머지는 버그를 발견하면 고치지 말고 기록만 합니다.
3. **버그 처리 순서:** 발견 → 이슈 목록 기록 → 담당자가 브랜치에서 수정 → `tests/test_core.py` , `09_golden_test.py` 통과 → `fix:` 커밋 → 다음 날 14:00 새 zip 배포. 이 기록이 그대로 Test Report 의 FAIL 기록이 됩니다.
4. **10/7 Freeze 이후**에는 버그 수정과 위 "부족한 부분"만 합니다. 새 기능은 넣지 않습니다.
5. **Class·BBox 기준서는 본 검수(10/8) 전에 확정**합니다. 6명이 같은 기준으로 판단해야 합니다.

버그 이슈 목록 양식

번호	발견자	증상	재현 방법	원인	조치	담당	재시험
FAIL-01	홍길동	Zoom 후 저장하면 BBox 위치가 어긋남	200% 확대 → 새 BBox → 저장 → 재실행	좌표 변환 누락	scale 반영	좌표	PASS

Git 커밋 메시지 규칙

```
feat: configs/classes.yaml 에서 Class 목록 읽기
fix: Zoom 상태에서 새 BBox 저장 위치 오류 수정
docs: Class 기준서 작성
# 금지: update / 수정 / 최종 / asdf
```

4. 역할별 산출물 담당 (라벨링 150장과 별도)

역할	맡을 산출물	마감
PM	☐ 4 Project Baseline ☐ 2 Git 저장소·커밋 규칙 ☐ 3 FINAL 생성과 캡처 증빙	Baseline·Git 10/7 FINAL 10/19
GUI	☐ 1 실행 캡처 3장·시연 영상 ☐ 10 README 정리 (문서 위치 안내) ☐ 화면 버그 담당	캡처 10/16 영상·README 10/19
좌표	☐ 6 BBox 기준서 (팀 GOOD/BAD 예시 이미지 포함) ☐ 좌표·Zoom 버그 담당	10/8 본 검수 전
저장	☐ `classes.yaml` 연동 ☐ FINAL 평면 내보내기 ☐ Manifest 내보내기 (`manifests/dataset_manifest.csv`)	yaml 10/7 나머지 10/14
QA	☐ 9 Test Report (Golden · Pilot · Acceptance, FAIL 기록 포함) ☐ Validation 운영과 이슈 목록 관리	Golden 10/7 완성 10/19

문서	☐ 5 Class 기준서 ☐ 8 QA Summary ☐ 7 Manifest 확인 ☐ 11 교과 8 Handoff	Class 기준 10/7 나머지 10/19

5. 오늘(10/6) PM 체크

☐ 전원 설치(`bash setup_wsl.sh`) 후 실제 데이터로 열기 확인 (전원)

☐ Git 저장소 생성, `.gitignore` 확인 (데이터·settings.json 제외) (PM)

☐ 첫 커밋 `feat: 라벨링 프로그램 기본 구조` (PM)

☐ 이슈 목록 만들기 (공유 문서 또는 GitHub Issues), 위 양식 공유 (PM·QA)

☐ 역할별 산출물 담당표 공지 (4장) (PM)

☐ RAW 해시 기록, Manifest 생성, RAW 사전 Validation (PM·문서·QA)

☐ 팀 실행 가이드 체크리스트 S1 항목 진행 (전원)

6. 일정에 맞춘 산출물 흐름

날짜	라벨링	산출물
10/6 (S1)	설치, 실제 데이터 연결	Git 저장소, 이슈 목록, RAW 해시·Manifest
10/7 (S2)	Pilot 30장	Golden Test, Class·BBox 기준서 초안, `classes.yaml`, Baseline, **Freeze**
10/8~10/14	본 검수 1인 150장	Pilot 결과·FAIL 기록 → Test Report, 내보내기 코드
10/15~10/16	교차검수 100%, PASS 표본	실행 캡처 3장, QA 오류 집계
10/19	FINAL 생성	FINAL 캡처, Acceptance Test, QA Summary, Handoff, README, 시연 영상
10/20~10/21	-	최종 점검 (아래 체크리스트), 리허설, 발표

7. 최종 제출 체크리스트 (산출물 예시 기준)

1. 라벨링 프로그램

☐ 프로그램이 정상 실행된다

☐ Load / TXT Load / BBox 수정 / 저장 / Reload / Validation 정상

☐ 실행 화면 캡처 2~3장 준비

☐ 발표 시 직접 시연 가능

2. 소스코드

- ☐ 실제 Python 소스코드를 Git 에 올렸다
- ☐ 기능별로 코드가 구분되어 있다
- ☐ requirements.txt 가 있다
- ☐ configs/classes.yaml 이 있다
- ☐ 기능 개발·수정 과정의 Commit 이력이 있다

3. FINAL YOLO 라벨 증빙

- ☐ final/images 900장, final/labels 900개
- ☐ 이미지·TXT 파일명 Pair 일치
- ☐ final 폴더 구조·파일 수 캡처
- ☐ 대표 검수 완료 화면 3~5장, 대표 TXT 2~3개

4. Project Baseline

- ☐ 공통 작업 기준, 작업자·검수자 역할
- ☐ WORK / FINAL 저장 위치, Git 작업 기준, 완료 기준

5. Class 기준서

- ☐ Class 0~6 의미, 포함 / 제외 기준
- ☐ 애매한 경우 REVIEW 기준, Class 4 사용 안 함 기준

6. BBox 기준서

- ☐ 객체 외곽 밀착 기준, 이미지 경계 객체 기준
- ☐ 겹친·작은·가려진 객체 기준, REVIEW 기준

7. Dataset Manifest

- ☐ 900장 작업 상태 전부 기록
- ☐ 출처·기존 Split·작업자·상태·QA 상태·REVIEW 사유

8. QA Summary

- ☐ Validation 결과와 오류 종류·수량
- ☐ Human QA 결과, 최종 FAIL 0 · REVIEW 0

9. Test Report

- ☐ Golden Test 결과
- ☐ Pilot Test 결과와 오류·수정·Retest 기록
- ☐ Final Acceptance Test 결과

10. README

- ☐ 소개·설치·실행·사용법·폴더 구조
- ☐ Class / BBox / Manifest / QA / Test 문서 위치, FINAL 위치, 주의사항

11. 교과 8 Handoff

☐ FINAL 수량, YOLO 형식, Class 정보

☐ 최종 QA 결과, Manifest·기준서 위치, FINAL 내부 저장 위치

최종 Git 저장소 구조 (목표)

```
kimchi_labeler/
 ├──── main.py, run.sh, setup_wsl.sh, requirements.txt
 ├──── README.md                    ← 10
 ├──── src/                         ← 1~2  (gui/, session.py, yolo_io.py, validator.py ...)
 ├──── configs/classes.yaml         ← 2
 ├──── scripts/  tests/
 ├──── docs/
 │     ├──── project_baseline.md     ← 4
 │     ├──── class_guide.md          ← 5
 │     ├──── bbox_guide.md           ← 6
 │     └──── subject08_handoff.md    ← 11
 ├──── manifests/dataset_manifest.csv ← 7
 └──── reports/
       ├──── qa_summary.md           ← 8
       └──── test_report.md          ← 9

# Git 에 올리지 않음: data/raw, data/work_*, data/final (images 900 / labels 900) ← 3
```